

Numpy basics

1. What is NumPy?

NumPy(Numerical Python) is a fundamental library for Python numerical computing. It provides efficient multi-dimensional array objects and various mathematical functions for handling large datasets making it a critical tool for professionals in fields that require heavy computation.

```
import numpy as np  
  
a = np.array([1, 2, 3])  
print(a)
```

0

Some important points about Numpy arrays:

- We can create an N-dimensional array in Python using `numpy.array()`
- The array is by default Homogeneous, which means data inside an array must be of the same Datatype.
- Element-wise operation is possible.
- Numpy array has various functions, methods, and variables, to ease our task of matrix computation.
- Elements of an array are stored contiguously in memory. For example, all rows of a two-dimensioned array must have the same number of columns. A three-dimensional array must have the same number of rows and columns on each card.

2. Why is NumPy faster than Python lists?

Arrays are stored in contiguous memory locations, enabling faster computations than Python lists.

Python Lists

1. **Element Overhead:** Lists in Python store additional information about each element, such as its type and reference count. This overhead can be significant when dealing with a large number of elements.
2. **Datatype:** Lists can hold different data types, but this can decrease memory efficiency and slow numerical operations.
3. **Memory Fragmentation:** Lists may not store elements in contiguous memory locations, causing memory fragmentation and inefficiency.
4. **Performance:** Lists are not optimized for numerical computations and may have slower mathematical operations due to Python's interpretation overhead. They are generally used as general-purpose data structures.
5. **Functionality:** Lists can store any data type, but lack specialized NumPy functions for numerical operations.

Numpy Arrays

1. **Homogeneous Data:** NumPy arrays store elements of the same data type, making them more compact and memory-efficient than lists.
2. **Fixed Data Type:** NumPy arrays have a fixed data type, reducing memory overhead by eliminating the need to store type information for each element.
3. **Contiguous Memory:** NumPy arrays store elements in adjacent memory locations, reducing fragmentation and allowing for efficient access.

4. **Array Metadata:** NumPy arrays have extra metadata like shape, strides, and data type. However, this overhead is usually smaller than the per-element overhead in lists.
5. **Performance:** NumPy arrays are optimized for numerical computations, with efficient element-wise operations and mathematical functions. These operations are implemented in C, resulting in faster performance than equivalent operations on lists.

3. What is an ndarray?

ndarray is a short form for N-dimensional array which is an important component of NumPy. It allows us to store and manipulate large amounts of data efficiently. All elements in an ndarray must be of the same type, making it a homogeneous array. This structure supports multiple dimensions, which makes it ideal for handling complex datasets like those used in scientific computing or data analysis. With this, we can perform fast and memory-efficient operations on data.

```
import numpy as np

arr1 = np.array([1, 2, 3, 4, 5])

arr2 = np.array([[1, 2, 3], [4, 5, 6]])

arr3 = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
print(arr1)
print(arr2)
print(arr3)
```

Output:

```
[1 2 3 4 5]
[[1 2 3]
 [4 5 6]]
[[[1 2]
 [3 4]]
 [[5 6]
 [7 8]]]
```

4. Difference between array and list

List	Array
Can consist of elements belonging to different data types	Consists of elements belonging to the same data type only
No need to explicitly import any module	Must explicitly import the array (or NumPy) module
Cannot directly handle arithmetic operations	Can directly handle arithmetic operations
Preferred for a shorter sequence of data items	Preferred for a longer sequence of data items
More flexible; easy addition and deletion of elements	Less flexible; addition and deletion done element-wise
Entire list can be printed without explicit looping	Loop is usually required to access or print elements
Consumes more memory due to flexibility	More memory-efficient and compact
Nested lists can be of variable size	Nested arrays must be of the same size
Provides many built-in methods like count(), sort(), max(), min(), sum(), index(), append(), remove() etc.	Requires importing proper modules to perform similar operations
No import required to use list functions	Need to import modules to use array operations

5. What is broadcasting?

Broadcasting in NumPy allows us to perform arithmetic operations on arrays of different shapes without reshaping them by automatically adjusting the smaller array to match the larger array's shape by replicating its values along the necessary dimensions. This makes element-wise operations more efficient by reducing memory usage and eliminating the

need for loops.

Lets see an example:

```
import numpy as np

a = np.array([[1, 2, 3], [4, 5, 6]])
x = 10
print(a + x)
```

Output

```
[[11 12 13]
 [14 15 16]]
```

Broadcasting Rules:

1. Arrays are compared **from right to left** (trailing dimensions).
2. Two dimensions are compatible if:
 - They are equal, or
 - One of them is **1**
3. If dimensions are incompatible → **ValueError**